

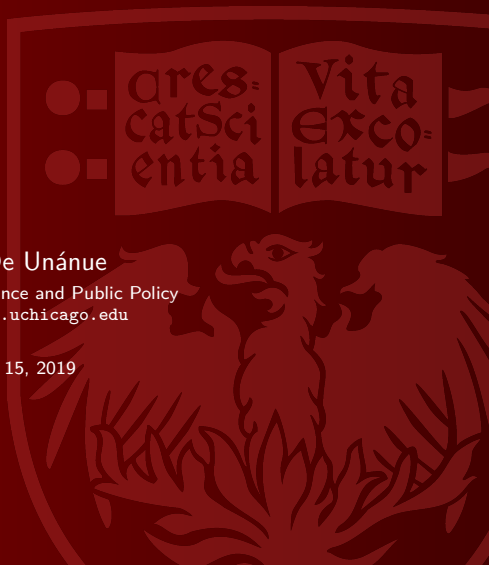
Featurizer

More features for everyone!

Adolfo De Unánue

Center for Data Science and Public Policy
<http://dsapp.uchicago.edu>

January 15, 2019



- One step aggregation to `as_of_dates`

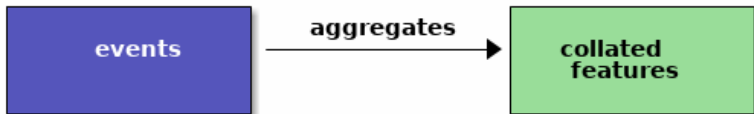


Figure: Collate does an *one step aggregation* of the events (plus imputation)

- Confusing / Non-intuitive semantics
 - The *meaning* (originally) refers to the aggregated variable
 - The `aggregate` is numeric, even if the variable was categorical
 - Then `categorical` is just a filter over the aggregate function
 - Weird use of `max`
 - Incorrect interpretation of `avg`
 - *encoding* is defined as a *ad-hoc* implementation of `one`

What do we want to achieve?



- Expand the features available to triage
- Improve `collate`'s semantics
- Simplify the way to adding new features
- Add governance to the features
 - Feature metadata
 - Short names
 - Descriptions
 - *Some stats* (Not implemented yet)

What did we do?



- We implemented the **DFS algorithm** (2015) in a mix of Python / PostgreSQL as the feature generator
- We added some triage specific functionality
 - time intervals
 - feature groups

How DFS works? : Generalities



- **DFS** = *Deep feature synthesis*
- Recursive algorithm
 - It traverses the tables through the relationships between tables
 - A slightly modification of a BFS for graph traversal
- Originally built for MySQL
 - No temporal handling at the beginning
- Currently, the open version works in `python`

How DFS works?: *Backward* and *Forward* relationships



How DFS works?: Transformers, Aggregators and Direct Transfers



How DFS works? Graphically

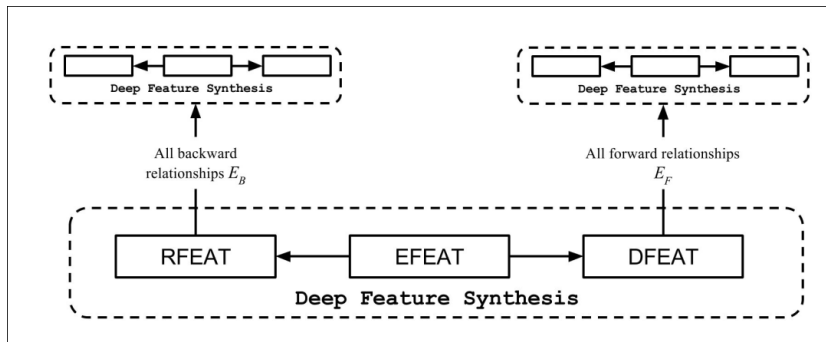


Figure: Image from *Deep Feature Synthesis: Towards Automating Data Science Endeavors*

Algorithm 1 Generating features for target entity

```
1: function MAKE_FEATURES( $E^i, E^{1:M}, E_V$ )
2:    $E_V = E_V \cup E^i$ 
3:    $E_B = \text{BACKWARD}(E^i, E^{1:M})$ 
4:    $E_F = \text{FORWARD}(E^i, E^{1:M})$ 
5:   for  $E^j \in E_B$  do
6:     MAKE_FEATURES( $E^j, E^{1:M}, E_V$ )
7:      $F^j = F^j \cup \text{RFEAT}(E^i, E^j)$ 
8:   for  $E^j \in E_F$  do
9:     if  $E^j \in E_V$  then
10:      CONTINUE
11:     MAKE_FEATURES( $E^j, E^{1:M}, E_V$ )
12:      $F^i = F^i \cup \text{DFEAT}(E^i, E^j)$ 
13:    $F^i = F^i \cup \text{EFAT}(E^i)$ 
```

- Implements the **DFS algorithm** in python to generate sql
- Supports a bigger set of *transformations* and *aggregations* than the current DFS
- Tracks the features dependencies
- Generates SQL
 - At this moment as a chain of CTEs
 - *Planned*: Allow the user to specify Temporary tables / tables instead of the CTEs
 - * In that way you can achieve better performance (*not implemented yet*)
- Executes the SQL
 - This is just for testing, when incorporated to triage it will use architect

- The idea is *automatically* generate features *for* a specific target
- The function's set is applied *globally*
- You can control the functions to be applied through *whitelists* and *blacklists*
- The intervals are *global* too
 - The reason behind this is a conceptual one: the config file *defines* the structure of the *input* tables no the *way to transform them*
- The other knob is `max_depth`

_class_diagram.png

```
./images/featurizer\unhbox \voidb@x \penalty \@M \hskip \z@skip
```

How to use it?



- YAML configuration file
- Example python code:

```
featurizer = Featurizer('test.yaml')
```

```
featurizer.query # shows the query ...
```

```
df = featurizer.to_dataframe() # ... or gets you a dataframe
```





- The SQL generation is quite dumb, there is no *pruning*
 - Remember there is a *double* limitation in PostgreSQL: (a) size of the row, and (b) number of columns
- Solve the column limit in PostgreSQL
- Unit tests
- Maybe we could simplify the configuration file *if* we specify the DB connection and use *metadata* magic
- Avoid stacks that don't make sense
- Store the Features metadata table (for *data lineage*)